Лабораторная работа №3
Работа с файлами и каталогами. FHS

Введение

Из этой лабораторной работы вы узнаете, что такое стандарт FHS, почему «всё есть файл», и познакомитесь с необычными видами файлов в Linux. Вы узнаете, как устроена файловая система Linux изнутри и с помощью каких команд наиболее эффективно управлять объектами файловой системы.

Основные отличия в работе с файлами (Windows и Linux)

Всё есть файл

Первым существенным отличием является то, что в Linux файлы это не то, чем они кажутся. Файлы используются не только для хранения информации на диске. С их помощью реализованы, например, каталоги и ссылки. А еще интерфейс доступа к файлам используется для взаимодействия с устройствами, каналами и сокетами.

Такой подход по обеспечению доступа ко всем возможностям операционной системы через унифицированный интерфейс файловой системы получил название «всё есть файл». Этот интерфейс, конечно, может быть не всегда оптимальным для решения поставленной задачи, зато он всегда одинаковый. Для примера рассмотрим каталог `/proc`. В эту точку монтируется виртуальная файловая система, которая предоставляет доступ к информации о системе и её процессах, поэтому с помощью команды `cat /proc/meminfo` можно увидеть параметры использования памяти.

Если в Linux придерживаются концепции «всё есть файл», то в Windows следуют концепции «всё есть объект», то есть у всех системных ресурсов файлов, каталогов, аппаратных устройств есть свойства и методы, как в объектно ориентированном программировании.

Единое пространство имен

Вторым отличием от Windows является единое пространство имен файловой системы. Если в Windows каждому блочному устройству назначается собственная буква и абсолютный путь к локальному файлу имеет вид `C:\pagefile.sys`, то в *nix подобных системах все диски встраиваются в единое пространство имен, которое начинается с косой черты `/` (корня).

Например, в операционной системе Astra Linux при подключении внешнего usb накопителя его файловая система монтируется в точку `/run/user/<UID>/media/<ID устройства>` с помощью модуля FUSE (*от англ. filesystem in userspace — «файловая система в пользовательском пространстве»*). Более того, любые

каталоги, например, `/home` или `/var`, могут физически располагаться на других устройствах, в том числе сетевых, и монтироваться средствами операционной системы.

Подход к хранению файлов программ

Третьим отличием является подход к хранению файлов. В системе Windows служебные файлы операционной системы находятся в каталоге `C:\Windows`, а файлы приложений в каталоге `C:\Program Files`, причем для каждого приложения создается отдельная папка, в которой хранятся все его файлы, организованные любым способом на усмотрение разработчиков приложения.

В Linux для организации файлов придерживаются так называемого стандарта иерархии файловых систем (Filesystem Hierarchy Standard, FHS), который унифицирует размещение файлов по их назначению. Например, исполняемые файлы приложений должны находиться в каталогах `/bin` и `/sbin` *от англ. binary - бинарные*), конфигурационные файлы находятся в папке `/etc` *от англ. «et cetera» - прочие*), а изменяемые файлы, такие как журналы, должны находиться в каталоге `/var` *от англ. variable - изменчивый*).

Зависимость от регистра символов

Четвертое отличие состоит в том, что файловые системы Linux традиционно учитывают регистр символов, то есть в одном каталоге одновременно могут находиться файлы `test.txt`, `Test.txt` и `TEST.txt`, которые будут считаться совершенно разными файлами.

Если же примонтировать в Linux диск, отформатированный в NTFS или exFAT, то мы сможем создать только один из этих файлов и обращаться к нему по любому из предложенных вариантов именования, как в Windows. То есть эта особенность является особенностью файловой системы, а не операционной.

Подход к использованию ссылок

Рассмотрим пятое отличие, которое заключается в использовании ссылок.

Несмотря на то, что команда `mklink` для создания ссылок появилась в системе Windows еще со времен NT, большинство пользователей Windows ограничиваются использованием ярлыков, которые представляют собой обычные файлы с расширением `*.lnk` и содержат настройки для быстрого доступа к файлам и папкам. Полным аналогом lnk файлов в Linux являются файлы `*.desktop`.

В системе Linux жесткие (hard link) и мягкие ссылки (symbolic link) получили широкое распространение, поэтому изменения в одном файле могут отражаться сразу в нескольких местах, что сначала может сильно сбивать с толку.

Например, с помощью команд `stat /run/cups/cups.sock` и `stat /var/run/cups/cups.sock` вы можете убедиться, что эти файлы указывают на айноду *от англ. information node, inode — информационный узел*) с одним и тем же номером, а все потому, что папка `/var/run` является мягкой ссылкой на папку `/run`, что можно проверить командой `ls -l /var/run`. Чуть позднее мы узнаем, что сделано так было для обеспечения обратной совместимости.

Еще интереснее, когда на один и тот же файл указывают сразу несколько жестких ссылок. Например, с помощью команд `stat /usr/bin/gunzip` и `stat /usr/bin/uncompress` вы можете легко убедиться, что это действительно две разные жесткие ссылки, которые указывают на одну и ту же айноду, поэтому до тех пор, пока мы не удалим обе ссылки, этот файл будет физически оставаться на диске.

Прямая косая черта в качестве разделителя

В завершение напомним, что в Linux в качестве разделителя между именами объектов файловой системы используют символ прямой косой черты «/» (слэш), например, `/etc/hosts`. В Windows можно использовать слэш, но более традиционным считается все же использование обратной косой черты « » (бэкслэш), например, `C:\Windows\System32\drivers\etc\hosts`.

Устройство файловой системы Linux

Корневая файловая система и стандарт FHS

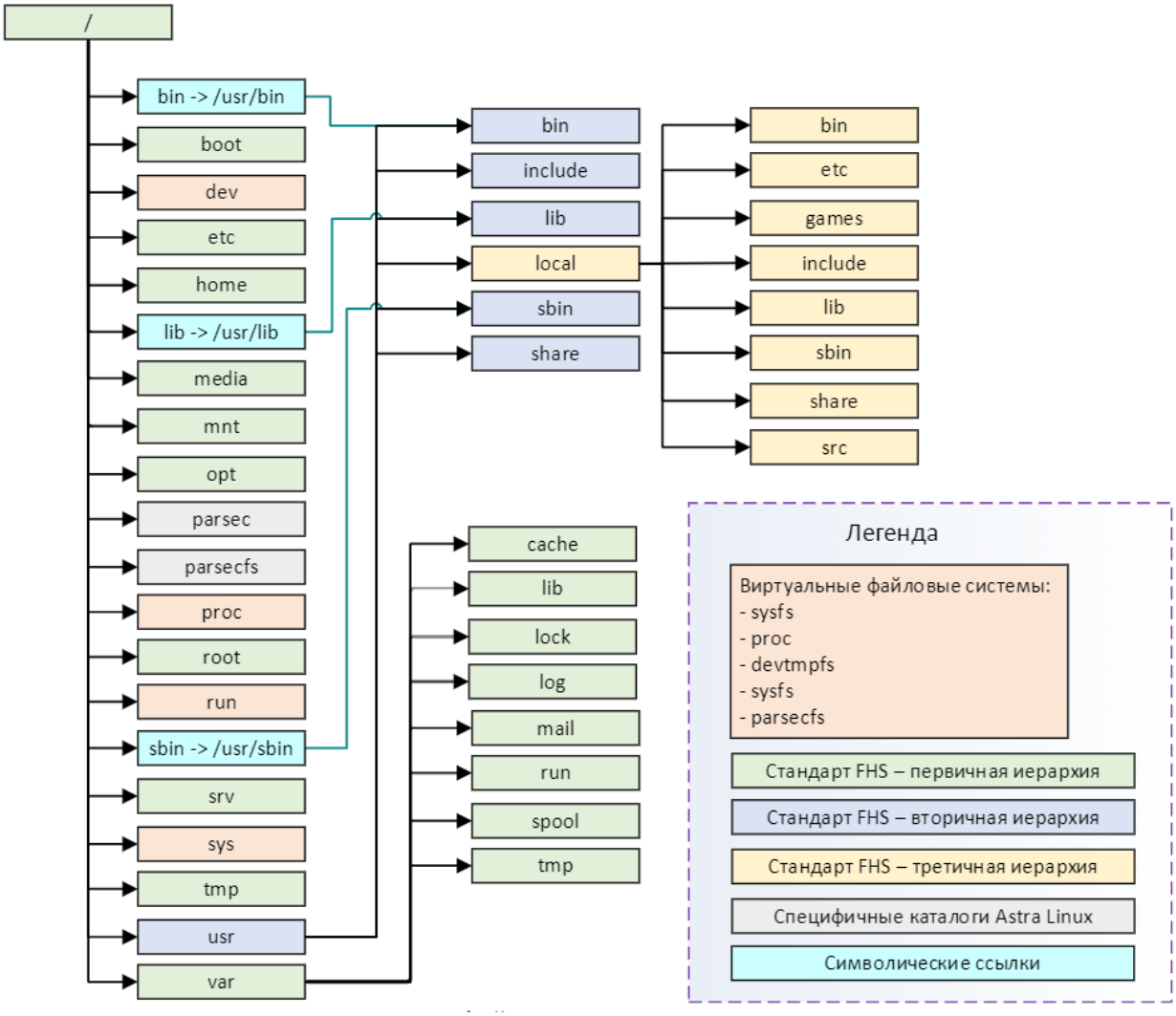
Системный диск подключается в Linux непосредственно к операционной системе, и точкой монтирования выступает та самая косая черта, которую называют еще корнем, а файловую систему системного диска, соответственно, называют корневой файловой системой. Именно к папкам корневой файловой системы в дальнейшем монтируются файловые системы других устройств.

Для того чтобы мы всегда знали, в каких каталогах искать нужные нам файлы, разработчикам потребовалось упорядочить структуру корневой системы, и они создали единый стандарт файловой системы (Filesystem Hierarchy Standard, FHS). Это очень странный стандарт, которого придерживаются практически все, но которому в полной мере не следует никто.

Причина такого положения вещей заключается в том, что некоторые идеи FHS уже устарели, но сообщество опасается делать резкие движения и продолжает двигаться по инерции.

Поэтому, изучая FHS, некоторые моменты лучше принять «как есть», а что то станет понятным только в сугубо историческом контексте. А если вы думаете, что в структуре

каталогов Windows нет подобных атавизмов, то вы просто еще не смотрели на нее глазами разработчиков!



Корневая файловая система в Astra Linux

Рассмотрим основные каталоги системы:

- `/` это корневой каталог системы, в эту точку монтируется системный диск.
- `/boot` содержит конфигурационные файлы загрузчика GRUB, образы ядра системы, файлы временной файловой системы initrd, используемые ядром Linux при начальной загрузке.
- `/dev` это каталог псевдофайловой системы devtmpfs, который содержит файлы устройств (*от англ. device - устройство*). Этот каталог является примером концепции «всё есть файл», поэтому в Windows подобного каталога не существует.

Все устройства можно разделить на две группы:

- **Символьные устройства** обеспечивают последовательный поток ввода/вывода. Например, мышь является символьным устройством, и каждое нажатие кнопки приводит к отправке символа в файл `/dev/input/mouse0`, в чем можно убедиться, если сделать cat этого файла.

- **Блочные устройства** предоставляют произвольный доступ к данным. Например, жесткий диск является блочным устройством, так как обеспечивает чтение/запись данных в произвольном порядке.

- `/media` точка для автоматического монтирования сменных носителей, таких как FDD, CD ROM, DVD ROM. В эту категорию не попадают флешки напомним, что внешние USB носители монтируются в точку `/run/user/<UID>/media/` средствами модуля FUSE (*от англ. filesystem in userspace — «файловая система в пользовательском пространстве»*).

- `/mnt` точка для монтирования внешних носителей вручную (*от англ. mount - монтировать*).

- `/proc` в эту точку монтируется виртуальная файловая система procfs, с помощью которой можно получать информацию о состоянии ядра операционной системы и запущенных процессах (*от англ. process - процесс*). Например, команда `ps` берет свои данные именно через файлы procfs.

Отметим, что некоторыми файлами procfs можно манипулировать, чтобы изменять конфигурацию ядра. Например, если записать новое значение в файл `/proc/sys/kernel/hostname`, то можно изменить имя хоста «на лету».

- `/sys` в эту точку монтируется виртуальная файловая система sysfs, которая является новой реализацией procfs и постепенно заменяет ее функции (*от англ. system - система*).

- `/run` каталог содержит pid файлы процессов, файлы временных сокетов и другие данные, описывающие систему с момента ее загрузки. Каталог размещается в TMPFS, поэтому очищается автоматически при перезагрузке системы.

Примечание

Ранее для тех же целей использовался каталог `/var/run`, который все еще остался в системе для обеспечения обратной совместимости, но является теперь символической ссылкой на каталог `/run`.

- `/bin` содержит бинарные файлы стандартных утилит Linux, которые должны быть доступны всем пользователям системы (*от англ. binaries - двоичные файлы*). В системе Windows такие файлы имеют расширение `*.exe` и находятся в каталоге `C:\Windows\System32`. Каталог является мягкой ссылкой на каталог `/usr/bin`.

- `/sbin` каталог содержит набор системных утилит, которые необходимы для загрузки системы и ее восстановления в случае сбоя, например, init, iptables, ifconfig (*от англ. system binaries - системные двоичные файлы*). Запускать эти утилиты имеет право только root. Каталог является мягкой ссылкой на каталог `/usr/sbin`.

- `/lib` в этом каталоге находятся динамические библиотеки, которые нужны для работы приложений из каталогов `/bin/` и `/sbin` *от англ. library - библиотека*). Каталог является мягкой ссылкой на каталог `/usr/lib`.

- `/usr/bin`, `/usr/sbin` и `/usr/lib` каталоги, на которые ссылаются `/bin`, `/sbin` и `/lib`. Причину, по которой эти каталоги существуют в двух экземплярах, можно понять только в контексте развития вычислительной техники.

Давным давно, когда жесткие диски были размером всего в пару мегабайт, разработчики операционной системы Unix довольно быстро подошли к этому пределу, и им пришлось перенести часть файлов из каталогов bin, sbin и lib на пользовательский диск (поэтому точка монтирования и называется `/usr`, от слова user). Критерий разнесения файлов по каталогам был прост те файлы, которые не требовались системе до монтирования пользовательского диска, переносились в каталог `/usr`.

В дальнейшем размер дисков кратно увеличился, и это решение потеряло актуальность, но для обеспечения обратной совместимости каталоги оставили и сделали на них мягкие ссылки. Вот и получается, что этому атавизму уже более 35 лет. И в ближайшее время вряд ли что то изменится.

- `/etc` содержит конфигурационные файлы операционной системы и всех ее служб (от лат. «et cetera» «и другие», «и тому подобное»). Данный каталог можно сравнить с **реестром Windows**, который был разработан для преодоления ограничений FAT16, связанных с отсутствием возможности разграничения прав доступа и медленным доступом к большому количеству маленьких INI файлов с настройками приложений.
- `/var` содержит файлы, которые подвергаются наиболее частому изменению (*от англ. variable - переменный*):
- `/var/cache` каталог предназначен для хранения файлов кэша, генерация которых требует значительного количества вычислительных ресурсов и/или операций ввода вывода. Удаление этих файлов не должно приводить к потере данных.
- `/var/lib` содержит файлы, которые используются для передачи информации о состоянии приложения между его вызовами или для обмена информацией между несколькими экземплярами приложения, которые работают одновременно. В этом каталоге, например, находятся файлы службы SSSD после ее установки.
- `/var/lock` файлы блокировки, указывающие на занятость устройств и других ресурсов. Эти файлы необходимы для возможности использования одного и того же ресурса несколькими приложениями.
- `/var/log` каталог содержит файлы журналов.
- `/var/mail` каталог содержит почтовые ящики локальных пользователей в устаревшем формате mbox, который представляет собой обычный текстовый файл. Для отправки сообщений локальным пользователям достаточно, чтобы в системе была установлена утилита `sendmail`, а, если в системе настроен модуль pam_mail, то при входе в терминал пользователи будут получать уведомления «you have new mail». Локальная почта в какой то степени является аналогом Windows утилиты msg.exe.
- `/var/run` устаревший каталог, который оставлен для обратной совместимости и является теперь символической ссылкой на каталог `/run`.

- `/var/spool` задачи, ожидающие обработки (например, очередь печати или очередь входящих/исходящих писем).
- `/var/tmp` временные файлы, которые должны сохраняться между перезагрузками системы.
- `/opt` каталог предназначен для размещения дополнительных приложений, которые отклоняются от стандарта FHS и хранят все свои файлы в одном месте (*от англ. optional - дополнительный*).

Каждому приложению выделяется отдельный каталог `/opt/<package>`, внутри которого могут быть каталоги bin, lib, man и др. для хранения исполняемых файлов, библиотек и справочных страниц соответственно. Установка таких приложений сводится к копированию одной папки и может выполняться даже без использования пакетных менеджеров. Таким образом, катало `/opt` является ближайшим аналогом для каталога `C:\Program files` системы Windows.

- `/srv` каталог предназначен для хранения файлов сетевых служб (*от англ. service - служба*). Например, если хост является веб сервером, то в этом каталоге может размещаться папка www с файлами обслуживаемых веб сайтов.
- `/tmp` каталог, в котором хранятся временные файлы. Linux, в отличие от Windows, следит за временными файлами и регулярно очищает этот каталог.
- `/home` содержит домашние каталоги обычных пользователей, в которых хранятся личные файлы, настройки программ и прочее. В системе Windows для тех же целей предназначен каталог `C:\Users`.
- `/root` домашний каталог суперпользователя. И тут встает вопрос: почему каталог `/root` лежит в корне, а не в папке `/home`? Когда жесткие диски были небольших размеров, одного жесткого диска хватало впритык для хранения файлов операционной системы, поэтому каталог `/home` почти всегда монтировали с отдельного жесткого диска. Но чтобы суперпользователь не терял доступ к своему домашнему каталогу даже в тех случаях, когда пользовательский диск не подключен, его домашний каталог был вынесен отдельно в корень системы.
- `/usr` каталог предназначен для хранения прикладных программ и может быть смонтирован по сети. В нем размещаются общие файлы, которые могут совместно использоваться множеством компьютеров сразу, поэтому в нем не должно быть файлов, специфичных для конкретного хоста, а у пользователей не должно быть прав на запись в этот каталог. В Windows для получения того же эффекта мы можем подключить сетевую папку с архивом полезных приложений, не требующих установки.

Примечание

В корне Astra Linux есть еще два каталога, специфичных для этой операционной системы: parsec и parsecfs. Они связаны с подсистемой безопасности PARSEC, входящей в состав ядра ОС.

Организации хранения данных на диске

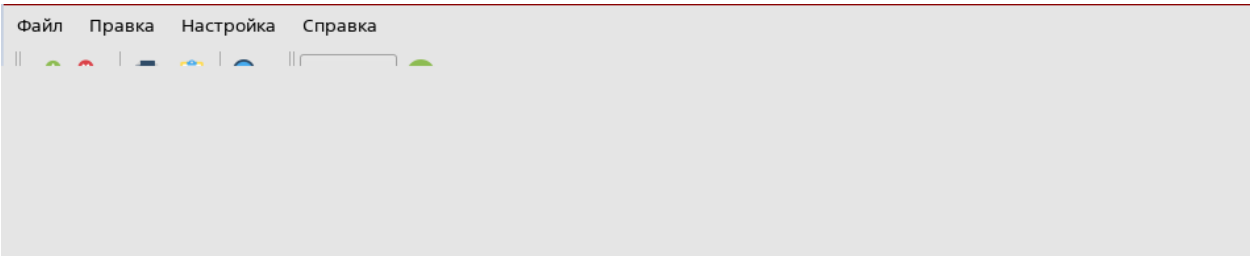
Файловые системы Windows и Linux используют немного разные подходы к организации хранения данных на диске, понимание которых позволит избежать проблем в будущем.

В Windows основной файловой системой является NTFS. Диск разбивается на кластеры, а для хранения метаинформации о файлах используется специальная база данных, которая называется главной файловой таблицей (Master File Table, MFT). Строкам этой таблицы или так называемым записям (MFT record) соответствуют файлы, а в столбцах таблицы хранятся значения конкретных атрибутов файлов.

По мере создания новых файлов размер MFT увеличивается, и файл этой базы может стать фрагментирован. Для того чтобы избежать деградации производительности системы в связи с фрагментированностью MFT, в момент форматирования диска система резервирует под MFT 12,5% дискового пространства и удерживает его до тех пор, пока не будет исчерпано все свободное пространство диска.

Наиболее популярной файловой системой Linux является EXT4. Диск разбивается на блоки, которые являются аналогом кластеров NTFS. А для хранения метаинформации о файлах используется структура данных, которая называется айнодой и является аналогом записей MFT.

Вывод команды `ls -l /home/sa/hello.txt` основан на данных из айноды:



1. Права на файл.
2. Количество жестких ссылок на файл.
3. Пользователь, являющийся владельцем файла.
4. Группа, являющаяся владельцем.
5. Размер файла в байтах.
6. Дата последнего изменения файла.
7. Полный путь к файлу.

А с помощью ключа `-i` утилита `ls` может показать номер индексного дескриптора:

```
sa@astra:~$ ls -i /home/sa/hello.txt
943173 /home/sa/hello.txt
```

Используя команду `stat`, можно получить ряд дополнительных атрибутов файла, хранящихся в его айноде. Одним из преимуществ команды `stat` в сравнении с `ls` является то, что она показывает права доступа к файлу в числовом формате, например, в приведенном ниже примере на файл hello.txt установлены права 0644:

Если подключить к виртуальной машине дополнительный диск объемом 32 Гб, затем создать на диске раздел с помощью утилиты `fdisk` и отформатировать этот раздел в ext4 утилитой `mkfs` с параметрами по умолчанию, то мы получим 8 млн. блоков по 4Кб и 2 млн. индексных дескрипторов.

```
$ lsblk
...
$ fdisk /dev/sdb
...
$ mkfs -t ext4 /dev/sdb1
...
Creating filesystem with 8388352 4k blocks and 2097152 inodes
...
```

По умолчанию размер блока составляет 4Кб, а количество дескрипторов определяется из расчета 1 дескриптор на каждые 16 384 байт дискового пространства, см. параметры blocksize и inode_ratio в конфигурационном файле `/etc/mke2fs.conf`. Вы можете изменить этот конфигурационный файл или явно задать размер блока и желаемое количество дескрипторов в момент форматирования диска с помощью ключей `-b` и `-N`:

```
$ mkfs -t ext4 -b 1024 -N 5000000 /dev/sdb1
...
Creating filesystem with 33553408 1k blocks and 5000000 inodes
...
```

Примечание

Файловая система XFS (X File System) позволяет динамически выделять блоки индексных дескрипторов, что снимает проблему их исчерпания. Если же вы используете более традиционную систему EXT4, то на этапе планирования вам следует определить оптимальные параметры диска и периодически следить за статистикой использования айнод.

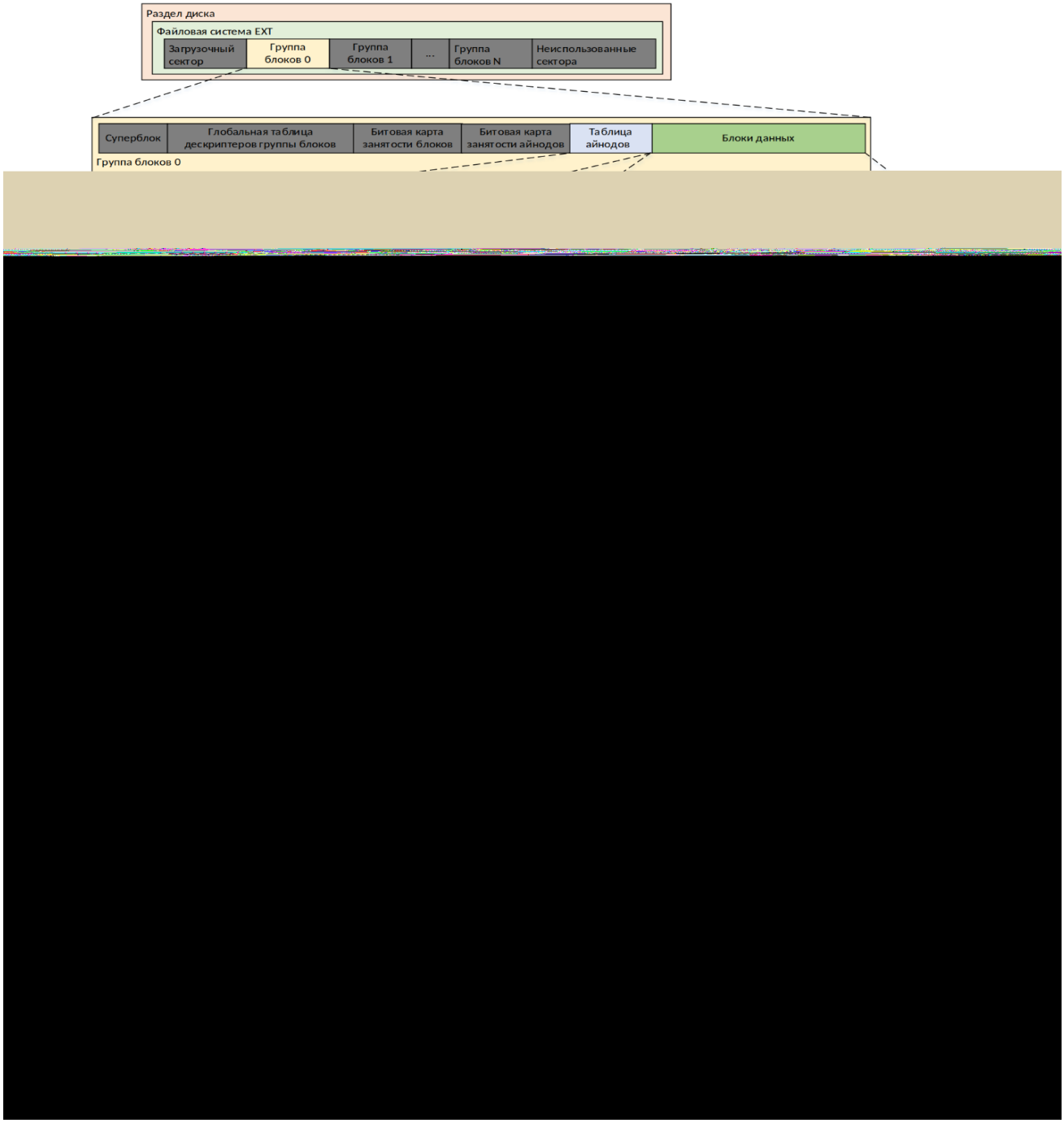
Узнать количество занятых и свободных айнод можно командой `df -i`.

```
sa@astra:~$ df -i
Файловая система Iнодов IИспользовано ICвободно IИспользовано% Смонтировано в
udev              240770           484    240286             1% /dev
tmpfs             253151           678    252473             1% /run
/dev/sda1         987360        226603    760757            23% /
tmpfs             253151             3    253148             1% /dev/shm
tmpfs             253151             4    253147             1% /run/lock
tmpfs             253151            19    253132             1% /sys/fs/cgroup
tmpfs             253151            10    253141             1% /run/user/107
tmpfs             253151            38    253113             1% /run/user/1000
```

Еще одним отличием является способ хранения имен файлов. Если в Windows имя файла является частью записи MFT, то в Linux хранение имен реализовано с помощью жестких ссылок.

Дело в том, что индексный дескриптор представляет собой структуру данных фиксированного размера (для ext4 это 256 байт), внутри которой может находиться только ограниченный набор атрибутов: тип файла, размер, дата создания/изменения, информация

о владельце и группе, базовые права доступа rwx, ну и, конечно же, ссылки на блоки данных.



Структура индексного дескриптора и схема адресации[3]

Поэтому дескриптор плохо подходит для хранения имен произвольной длины. Но для этого отлично подходят специальные файлы каталогов.

В файлы каталогов записываются так называемые жесткие ссылки, которые связывают имя файла с номером индексного дескриптора. У дескриптора таких ссылок может быть сразу несколько, но все они являются равнозначными, поэтому файл будет удален с диска только в том случае, если будут удалены все его жесткие ссылки.

Для того чтобы можно было легко принимать решение об удалении файла, внутри айноды есть атрибут, с помощью которого файловая система ведет подсчет количества жестких ссылок. При создании новой ссылки значение счетчика увеличивается на единицу, при удалении уменьшается, а если счетчик станет равен нулю, то файл будет удален.

Давайте рассмотрим процесс, как файловая система работает с айнодами. Если мы захотим прочитать файл, то система возьмет номер дескриптора из жесткой ссылки, считает метаинформацию о файле, выполнит проверку прав доступа и вернет содержимое файла, прочитав его с диска по ссылкам на блоки данных. В тех случаях, когда размер файла укладывается в 12 блоков, системе будет достаточно так называемых прямых ссылок. Для более крупных файлов есть несколько ссылок косвенной адресации, которые позволяют использовать блоки данных для хранения ссылок. Такой подход позволяет адресовать файлы размером до 16 терабайт при стандартном размере блока в 4 Кб. Структура индексного дескриптора и схема адресации блоков данных представлена на рисунке выше.

Типы индексных дескрипторов

В Linux существует семь типов айнод, пять из которых относят к специальным файлам, см. таблицу ниже.

Тип файла	**Обозначение в ls**	**Назначение**
Обычные файлы	**-**rwxrwxrwx	Содержат различные данные, например, тексты, картинки, архивы, скрипты, библиотеки, исполняемые файлы и т. д.
Каталоги	**d**rwxrwxrwx	Содержат жесткие ссылки, связывающие имена файлов с индексными дескрипторами.
Специальные файлы:		
Символические ссылки	**l**rwxrwxrwx	Предоставляют альтернативное имя для доступа к файлам, которые расположены в любом месте файловой системы, в том числе на других носителях (аналог mklink в Windows).
Блочные устройства	**b**rwxrwxrwx	Предоставляют интерфейс для взаимодействия с аппаратным обеспечением компьютера.
Символьные устройства	**c**rwxrwxrwx	
Каналы	**p**rwxrwxrwx	Обеспечивают в Linux работу механизмов межпроцессного взаимодействия.
Сокеты	**s**rwxrwxrwx	

Обычные файлы

К обычным файлам можно отнести все файлы с данными, например, тексты, картинки, архивы, скрипты, исполняемые файлы, библиотеки и т. д. Кратким обозначением обычных файлов является символ дефиса « ».

Откроем терминал и введем команду `ls -l`, знакомую нам по лабораторной работе **№2, где мы создали 2 файла script и hello.txt:**

Команда `cp <источник> <назначение>` копирует файлы и каталоги, где *источник* или несколько <*источников*> копируются в указанное <*назначение*>:

- `cp file1 file2` копирует файл *file1* в файл *file2*.
- `cp file1 destination/` копирует файл *file1* в каталог *destination*.
- `cp -r source/ destination` (`-r`, `--recursive`) копирует каталог source и все подкаталоги в каталог destination, если каталог destination существует. Если каталог не существует, то создаст его и скопирует **содержимое** каталога source.
- `cp -rT source destination/` (`-r`, `--recursive`; `-T`, `--no-target-directory`) копирует содержимое каталога source и все подкаталоги в каталог destination. Если каталога destination не существует, создаст его.
- `cp -r source/* destination/` копирует содержимое каталога source и все подкаталоги в каталог destination, если каталог destination существует. Если каталога не существует, выдаст ошибку.
- `cp -i file1 destination/` (`-i`, `--interactive`) копирует файл *file1* в каталог с запросом на подтверждение перезаписи. Каталог должен существовать.
- `cp -n file1 destination` ключ `-n` запрещает заменять существующие файлы.
- `cp -b file1 destination` ключ `-b` создает резервную копию файла.
- `cp slink destination` копирует не символическую ссылку slink, а оригинальный файл.
- `cp hlink destination` копирует не жесткую ссылку hlink, а оригинальный файл.
- `cp -P slink destination` копирует символическую ссылку именно как ссылку.
- `cp --preserve=links file1_hlink1 file1_hlink2_file destination` если file1_hlink1 и file1_hlink2 являются жесткими ссылками на один и тот же индексный дескриптор, то утилита скопирует исходный файл и создаст для него две жесткие ссылки в каталоге destination.
- `cp -p source/\* destination` копирует содержимое каталога source в каталог destination с сохранением информации о владельцах и прав доступа, хранящейся в индексном дескрипторе.
- `cp --preserve=timestamps file destination` копирует файл в каталог destination с сохранением временных меток оригинального файла.
- `cp -a source/* destination` копирует структуру файловой системы 1 в 1, включая временные метки, символические и жесткие ссылки.

Примеры использования:

Подготовка для демонстрации тестового каталога с набором файлов, ссылок и подкаталогом с файлами.

```
sa@astra:~$ mkdir -p ./test_cp/destination
sa@astra:~$ cd test_cp
sa@astra:~/test_cp$ mkdir ./source
```

```
sa@astra:~$ touch ./file{1..3}
sa@astra:~$ touch ./source/file{4..6}
sa@astra:~$ ln ./file1 ./hlink_file1
sa@astra:~$ ln -s $(realpath file2) ./slink_file2
sa@astra:~/test_cp$ tree
.
├── destination
├── file1
├── file2
├── file3
├── hlink_file1
├── slink_file2 -> /home/sa/file2
└── source
    ├── file4
    ├── file5
    └── file6
2 directory, 8 files
```

Копирование файла в файл

sa@astra:~/test_cp$ cp file1 file1_1 sa@astra:~/test_cp$ ls destination file1 file1_1 file2 file3 hlink_file1 slink_file2

Копирование файла в каталог назначения. Каталог должен существовать.

```
sa@astra:~/test_cp$ cp file1 destination
sa@astra:~/test_cp$ ls ./destination/
file1
```

Копирование каталога в каталог. Каталог должен существовать

```
sa@astra:~/test_cp$ cp -r source destination/
sa@astra:~/test_cp$ ls destination/
file1  source

sa@astra:~/test_cp$ cp -r source destination2/
sa@astra:~/test_cp$ ls destination2
file4  file5  file6

sa@astra:~/test_cp$ cp -r source destination2/
sa@astra:~/test_cp$ ls destination2
file4  file5  file6   source
```

Копирование содержимого каталога в каталог. Каталог должен существовать.

Способ 1.

```
sa@astra:~/test_cp$ cp -rT source destination/
sa@astra:~/test_cp$ ls destination/
file1  file4  file5  file6  source
```

Удаление скопированных файлов

```
sa@astra:~/test_cp$ rm ./destination/file{4..6}
sa@astra:~/test_cp$ ls destination/
file1  source
```

Способ 2.

```
sa@astra:~/test_cp$ cp source/* destination/
sa@astra:~/test_cp$ ls destination/
file1  file4  file5  file6  source
```

Копирование файла в каталог с запросом на подтверждение перезаписи. Каталог должен существовать.

```
sa@astra:~/test_cp$ cp -i file1 destination/
cp: переписать 'destination/file1'? y
sa@astra:~/test_cp$ stat destination/file
stat: не удалось выполнить stat для 'destination/file': Нет такого файла или каталога
sa@astra:~/test_cp$ stat destination/file1
 Файл: destination/file1
 Размер: 0             Блоков: 0          Блок В/В: 4096   пустой обычный файл
Устройство: 801h/2049d  Inode: 453120      Ссылки: 1
Доступ: (0644/-rw-r--r--)  Uid: ( 1000/      sa)   Gid: ( 1000/      sa)
Доступ: 2025-02-17 11:52:53.154355004 +0300
Модифицирован: 2025-02-17 11:55:51.450356010 +0300
Изменён: 2025-02-17 11:55:51.450356010 +0300
Создан: -
```

Создание резервной копии файла

```
sa@astra:~/test_cp$ cp -b file1 destination
sa@astra:~/test_cp$ ls destination
file1  file1~  source
```

Копирование символических ссылок

```
sa@astra:~/test_cp$ cp slink_file2 destination/
sa@astra:~/test_cp$ ls -li slink_file2
1075495 lrwxrwxrwx 1 sa sa 22 фев 17 11:51 slink_file2 -> /home/sa/test_cp/file2
sa@astra:~/test_cp$ ls -li destination/slink_file2
1075506 -rw-r--r-- 1 sa sa 0 фев 17 12:04 destination/slink_file2
sa@astra:~/test_cp$ cp -P slink_file2 destination/
sa@astra:~/test_cp$ ls -li destination/slink_file2
1075506 lrwxrwxrwx 1 sa sa 22 фев 17 12:05 destination/slink_file2 -> /home/sa/test_cp/file
2
```

Копирование жестких ссылок

```
sa@astra:~/test_cp$ rm -rf destination/file1
sa@astra:~/test_cp$ ls -li *file1
1075489 -rw-r--r-- 2 sa sa 0 фев 17 11:50 file1
1075489 -rw-r--r-- 2 sa sa 0 фев 17 11:50 hlink_file1
sa@astra:~/test_cp$ cp file1 hlink_file1 destination/
sa@astra:~/test_cp$ ls -li destination/*file1
1075505 -rw-r--r-- 1 sa sa 0 фев 17 12:01 destination/file1
1075507 -rw-r--r-- 1 sa sa 0 фев 17 12:01 destination/hlink_file1
sa@astra:~/test_cp$ rm -rf destination/*file1
sa@astra:~/test_cp$ cp --preserve=links file1 hlink_file1 destination/
sa@astra:~/test_cp$ ls -li destination/*file1
1075505 -rw-r--r-- 2 sa sa 0 фев 17 12:02 destination/file1
1075505 -rw-r--r-- 2 sa sa 0 фев 17 12:02 destination/hlink_file1
```

Сохранение атрибутов (в этом примере временных меток) при копировании

```
sa@astra:~/test_cp$ cp file2 destination/file2_1
sa@astra:~/test_cp$ cp --preserve=timestamps file2 destination/file2_2
sa@astra:~/test_cp$ ls -li file2
1075490 -rw-r--r-- 1 sa sa 0 фев 17 11:50 file2
sa@astra:~/test_cp$ ls -li destination/file2*
1075507 -rw-r--r-- 1 sa sa 0 фев 17 12:11 destination/file2_1
1075508 -rw-r--r-- 1 sa sa 0 фев 17 11:50 destination/file2_2
```

Полное копирование с сохранением всех атрибутов

```
sa@astra:~/test_cp$ cp -a source destination/source_backup
sa@astra:~/test_cp$ ls -l source
итого 0
- rw-r--r-- 1 sa sa 0 фев 17 11:51 file4
-rw-r--r-- 1 sa sa 0 фев 17 11:51 file5
-rw-r--r-- 1 sa sa 0 фев 17 11:51 file6
sa@astra:~/test_cp$ ls -l destination/source_backup
итого 0
-rw-r--r-- 1 sa sa 0 фев 17 11:51 file4
-rw-r--r-- 1 sa sa 0 фев 17 11:51 file5
-rw-r--r-- 1 sa sa 0 фев 17 11:51 file6
```

Команда mv

Команда mv перемещает (переименовывает) файлы и каталоги.

- `mv file destination` перемещает файл file в каталог destination.
- `mv directory destination` перемещает каталог directory в каталог destination.
- `mv file1 file2` переименовывает файл file1 в file2.
- `mv directory1 directory2` переименовывает каталог directory1 в directory2.

Примеры использования:

Подготовка для демонстрации тестового каталога с набором файлов и подкаталогами с файлами.

```
sa@astra:~$ cd
sa@astra:~$ mkdir test_mv
sa@astra:~$ cd test_mv
sa@astra:~/test_mv$ touch ./file{1..3}
sa@astra:~/test_mv$ mkdir destination
sa@astra:~/test_mv$ mkdir source
sa@astra:~/test_mv$ touch ./source/file{4..5}
sa@astra:~/test_mv$ tree
.
├── destination
├── file1
├── file2
├── file3
└── source
    ├── file4
    └── file5
2 directories, 5 files
```

Перемещение файла.

```
sa@astra:~/test_mv$ ls -il file1
154405 -rw-r--r-- 1 sa sa 0 фев 17 12:15 file1
sa@astra:~/test_mv$ mv file1 destination/
sa@astra:~/test_mv$ ls -il destination/file1
154405 -rw-r--r-- 1 sa sa 0 фев 17 12:15 destination/file1
```

Перемещение каталога

```
sa@astra:~/test_mv$ mv source destination/
sa@astra:~/test_mv$ tree
```

```
.
├── destination
│   ├── file1
│   └── source
│       ├── file4
│       └── file5
├── file2
└── file3
2 directories, 5 files
```

Переименование файла

```
sa@astra:~/test_mv$ ls -li file2
154406 -rw-r--r-- 1 sa sa 0 фев 17 12:15 file2
sa@astra:~/test_mv$ mv file2 file4
sa@astra:~/test_mv$ ls -li file4
154406 -rw-r--r-- 1 sa sa 0 фев 17 12:15 file4
```

Команда rm

Команда `rm` удаляет файлы и каталоги.

- `rm file` удаляет файл file.
- `rm -d directory` удаляет каталог directory, если он пуст.
- `rm -i file` перед удалением файла file запрашивает подтверждение у пользователя.
- `rm -I files` запрашивает подтверждение при удалении более 3 файлов.
- `rm -f file` удаляет файл file, игнорирует ключ `-i` и отсутствующие файлы и аргументы. Никогда не запрашивает подтверждения.
- `rm -Ir directory` рекурсивно удаляет каталоги и их содержимое. Запрашивает подтверждение.
- `rm -rf directory` **рекурсивно удаляет каталоги и их содержимое. Никогда не запрашивает подтверждения!!!**

Примеры использования:

Подготовка для демонстрации тестового каталога с набором файлов, ссылок и подкаталогом с файлами.

```
sa@astra:~$ cd
sa@astra:~$ mkdir -p test_rm/directory1/subdirectory
sa@astra:~$ cd test_rm/
sa@astra:~/test_rm$ mkdir directory2
sa@astra:~/test_rm$ touch file{1..3}
sa@astra:~/test_rm$ touch directory1/file{4..5}
sa@astra:~/test_rm$ touch directory1/subdirectory/file{6..7}
sa@astra:~/test_rm$ tree
.
├── directory1
│   ├── file4
│   ├── file5
│   └── subdirectory
│       ├── file6
│       └── file7
├── directory2
├── file1
├── file2
└── file3
3 directories, 7 files
```

Удаление простого файла

```
sa@astra:~/test_rm$ rm file1
```

Удаление простого файла с запросом на подтверждение

```
sa@astra:~/test_rm$ rm -i file2
rm: удалить пустой обычный файл 'file2'? y
sa@astra:~/test_rm$ rm -I file3
```

Удаление пустого каталога

```
sa@astra:~/test_rm$ rm directory2
rm: невозможно удалить 'directory2': Это каталог
sa@astra:~/test_rm$ rm -d directory2
```

Удаление непустого каталога

```
sa@astra:~/test_rm$ rm -d directory1/subdirectory
rm: невозможно удалить 'subdirectory': Каталог не пуст
sa@astra:~/test_rm$ rm -dr directory1/subdirectory
sa@astra:~/test_rm$ tree
.
├── directory1
│   ├── file4
│   └── file5
└── file3

1 directory, 3 files
```

Удаление непустого каталога с запросом на подтверждение

```
sa@astra:~/test_rm$ rm -drI directory1
rm: удалить 1 аргумент рекурсивно? n
sa@astra:~/test_rm$ rm -rI directory1
rm: удалить 1 аргумент рекурсивно? n
```

Удаление непустого каталога без запроса на подтверждение

```
sa@astra:~/test_rm$ rm -rf directory1
```

Команда find

Команда `find` ищет файлы в иерархии каталогов. Это мощное средство с большим количеством критериев поиска и возможностью совершать ряд действий с найденными файлами. Для получения полной справки используйте команду `man find`.

Синтаксис команды:

```
find [каталог] [критерий поиска] [действие]
```

Где:

- **каталог** это отправная точка поиска,

• **критерий поиска** можно указывать несколько критериев, применяя операторы условий:

Основные критерии поиска:

- `-name` поиск по имени с учетом регистра
- `-iname` поиск по имени без учета регистра,
- `-type` поиск по типу файла (f, d, l, b, c, p, s),
- `-size` поиск по размеру файла («+» больше, « » меньше, без знака точное совпадение, «c» байт, «k» Кбайт, «M» Мбайт, «G» Гбайт), например, «+1G», « 1M», «24K».
- `-empty` поиск пустых фалов,
- `-cmin` поиск по времени изменения в минутах, например, для поиска файлов, измененных за последний час « 60», для поиска всех файлов, кроме тех, что были изменены за последний час «+60»,
- `-atime` поиск файлов по времени доступа к ним в днях, например, «+30» поиск файлов, к которым не обращались за последние 30 дней,
- `-user` поиск файлов по владельцу,
- `-perm` поиск файлов по набору прав доступа, например, «755» или «777».

Основные операторы условий поиска:

- `-and` для объединения условий логическим «и»,
- `-or` для объединения условий логическим «или»,
- `-not` для инвертирования условия.

• **действие** что необходимо делать с каждым найденным файлом. Основные действия:

- `-delete` удаляет объекты, соответствующие результатам поиска,
- `-ls` выводит более подробные результаты поиска,
- `-print` применяется по умолчанию, если не указать другое действие, показывает полный путь к найденным файлам,
- `-exec` выполняет указанную команду в каждой строке результатов поиска:

```
-exec command {} \;
```

Где: `command` команда, например, `cp`, `{}` файл из результата поиска, а `\;` завершение команды.

Использование команды:

• `find . -name file` поиск файлов в текущем каталоге по имени или маске file.

• `find ./directory -name file` поиск файлов в каталоге directory по имени или маске file.

- `find . -size +1M` поиск файлов в текущем каталоге и подкаталогах с размером более 1МБ.
- `find . -size +1M -exec du {} \;` поиск файлов в текущем каталоге и подкаталогах с размером более 1МБ и вывод размеров этих файлов.
- `find ./directory –empty` поиск пустых файлов и каталогов в каталоге directory и подкаталогах.
- `find ./directory -empty -delete` поиск пустых файлов и каталогов в каталоге directory и подкаталогах и их удаление.
- `find . -cmin -60 -size +10k` поиск файлов и каталогов в текущем каталоге и подкаталогах, которые были изменены в течение прошлых 60 минут и у которых размер более 10КБ.
- `find . -name file\* -or -name \*txt` поиск файлов и каталогов в текущем каталоге и подкаталогах по маске «file*» ИЛИ «*.txt».
- `find . -name file\* -type d` поиск каталогов в текущем каталоге и подкаталогах по маске «file *».
- `find /etc -type f -exec grep -iH 'nameserver' {} \\;` поиск файлов в каталоге /etc, которые содержат подстроку nameserver.

Примеры использования:

Поиск файлов в текущем каталоге и подкаталогах по маске «test*.txt»

```
sa@astra:~$ find . -name test\*.txt
./test\.txt
./test4/test3.txt
./test4/test2.txt
./test4/test.txt
./test.txt
```

Поиск файлов в каталоге test4 и подкаталогах по маске «test*.txt»

```
sa@astra:~$ find ./test4 -name test\*.txt
./test4/test.txt
```

Поиск файлов в текущем каталоге и подкаталогах с размером более 1МБ.

```
sa@astra:~$ find . -size +1M
/.local/share/flyhelp/ru/.flyhelp/fts
./.mozilla/firefox/ipkecuu8.default-release/places.sqlite
...
```

Поиск файлов в текущем каталоге и подкаталогах с размером более 1МБ и вывод размеров этих файлов

```
sa@astra:~$ find . -size +1M -exec du {} \;
3492    ./.local/share/flyhelp/ru/.flyhelp/fts
5120    ./.mozilla/firefox/ipkecuu8.default-release/places.sqlite
3060    ./.mozilla/firefox/ipkecuu8.default-release/security_state/data.safe.bin
...
```

Поиск пустых файлов и каталогов в указанном каталоге и его подкаталогах

```
sa@astra:~$ find ./test -empty
/test/test2
./test/test4/test.txt
```

Поиск пустых файлов и каталогов в указанном каталоге и его подкаталогах и их удаление

```
sa@astra:~$ find ./test -empty -delete
sa@astra:~$ find ./test -empty
sa@astra:~$
```

Поиск файлов и каталогов в текущем каталоге и подкаталогах, которые были изменены в течение последних 60 минут

```
sa@astra:~$ find . -cmin -60
/.fly/startmenu/recents
./.fly/startmenu/recents/fly-term.desktop
./.fly/menutime
./.local/share/baloo/index-lock
./.local/share/baloo/index
./test_rm
./test_rm/file3
./test_rm/directory1
./test_rm/directory1/file5
./test_rm/directory1/file4
./.config/rusbitech
...
```

Поиск файлов и каталогов в текущем каталоге и подкаталогах, которые были изменены в течение последних 60 минут и у которых размер более 10КБ

```
sa@astra:~$ find . -cmin -60 -size +10k
./.local/share/baloo/index
./.cache/icon-cache.kcache
./.xsession-errors
```

Поиск файлов и каталогов в текущем каталоге и подкаталогах по маске «test*» ИЛИ «*txt»

```
sa@astra:~$ find . -name test\* -or -name \*txt
./test
./test/test                                                                    file.txt
./test/logo.txt
./test/-test.txt
./test/test1
./test/test1_link
...
```

Поиск каталогов в текущем каталоге и подкаталогах по маске «test*»

```
sa@astra:~$ find . -name test\* -type d
./test_rm
./test
./test4
./test_cp
./test_mv
```

Практические задания

Задание 1.

1. Найдите в директории /usr/share все файлы размером более 500 Кб.
2. Полученный в предыдущем пункте список сохраните в файл /tmp/search.list.
3. Выясните размер файла /tmp/search.list в килобайтах и сохраните это значение в файл /tmp/search size.txt.
4. Выясните количество строк в файле /tmp/search.list и запишите результат в файл /tmp/search size.txt, сохранив предыдущие данные.

Задание 2.

1. Создайте одной командой директорию /tmp/block6/task/1/2/3 и переместите в нее так же одной командой файлы /tmp/search.list и /tmp/search size.txt.
2. Создайте жесткую и символическую ссылки на файл /tmp/block6/task/1/2/3/search size.txt.
3. Выведите содержимое каталога /tmp/block6/task/1/2/3/ с получением айноды всех файлов.
4. Удалите созданный каталог /tmp/block6/task/1/2/3/ вместе с содержимым.

Вопросы

1. Какой каталог содержит конфигурационные файлы?
2. Можно ли в ОС Linux создать два файла /tmp/test.txt и /tmp/TEST.txt?
3. Какая максимальная длина имени файла в ОС Linux?
4. Каким символом в выводе команды `ls -l` помечаются обычные файлы?
5. Можно ли найти неименованный канал в структуре каталогов?
6. Каким ключом команды `ls` можно отобразить айноду?
7. Можно ли создать жесткую ссылку на файл, расположенный на другой файловой системе?
8. Путь «tmp/1/2.txt» является абсолютным или относительным?
9. Что будет при вызове команды touch с существующим файлом?